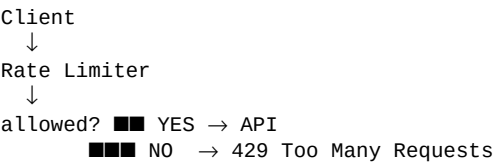


Rate Limiter — Practical System Design Notes

Hinglish • Node.js + Redis • API Protection • Interview-focused

1. Rate Limiter Kya Hai?

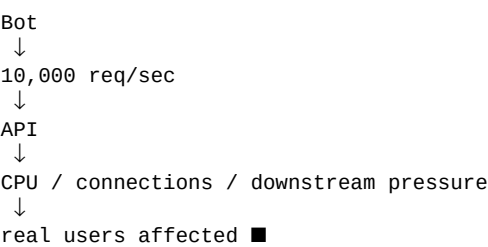
Rate limiter control karta hai ki ek client given time period mein kitni requests bhej sakta hai. Iska goal abuse control, fair usage, overload protection aur predictable service behavior hota hai.



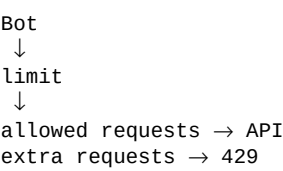
Simple mental model: 'Ek client system ko kitni speed se hit kar sakta hai?'

2. Rate Limiting Kyun?

Without rate limiting:



With rate limiting:



Rate limiting DDoS protection ka complete replacement nahi hai; large attacks ke liye upstream network/CDN/WAF/DDoS protection bhi needed ho sakta hai.

3. Kisko Limit Karna Hai?

Key	Example
IP	203.x.x.x → 100 req/min
User ID	user:123 → 60 req/min
API Key	key:abc → 1,000 req/hour
Endpoint	POST /login → stricter limit
Tenant	company:acme → 10,000 req/min

Real systems mein multiple dimensions combine ho sakte hain, e.g. per-user + per-IP + endpoint-specific limits.

4. Fixed Window Counter

Fixed window mein ek fixed time interval ke andar request count maintain kiya jata hai.

Limit = 100 requests / minute

10:00:00 ■■■■■■■■ 10:00:59
counter = 100 → allowed
101st → 429

10:01:00
counter resets → allowed again

Problem: Window boundary burst. Client 10:00:59 par 100 aur 10:01:00 par 100 requests bhej sakta hai, so short interval mein 200 requests aa sakti hain.

5. Sliding Window

Sliding window current request ke around previous rolling time interval ko consider karta hai. Ye fixed-window boundary burst ko better control kar sakta hai.

At 10:00:30
look back last 60 seconds

10:00:30 ←■■■■■■■ 09:59:30

Count requests in rolling window
→ if count >= limit → reject

Trade-off: implementation/memory complexity fixed counter se higher ho sakti hai. Exact algorithm can use logs, sorted sets, counters, or specialized stores.

6. Sliding Window Counter

Ek practical approximation current and previous fixed windows ke counters ko combine karke rolling behavior estimate kar sakti hai. Exact implementation requirements ke according choose hoti hai.

Interview mein difference yaad rakho: fixed window simple but boundary burst; sliding window more precise but more work.

7. Token Bucket

Token bucket mein bucket ki capacity hoti hai aur tokens fixed rate par refill hote hain. Har request ek token consume karti hai. Tokens available hain toh request allowed; nahi toh reject/delay.

Bucket capacity = 10
Refill = 5 tokens/sec

Initially: 10 tokens

Request → token consume
Request → token consume
...

No token → reject
or wait

Allows short bursts up to bucket capacity.

8. Token Bucket Kyun Popular Hai?

Token bucket average rate ko control karte hue controlled bursts allow kar sakta hai. APIs ke liye ye behavior often useful hota hai.

Normal traffic:
• • • • •
↓

tokens refill

Short burst:

.....



bucket capacity allows burst
then refill rate controls next requests

9. Leaky Bucket

Leaky bucket model incoming requests ko queue mein rakhkar roughly controlled processing rate se release karta hai. Iska focus smooth output rate par hota hai.

Incoming:

□ □ □ □ □ □ □ □ □ □

Queue

■■■■■■■■

Processor

→

→

→

→

Output becomes smoother

Token bucket burst allowance ke liye natural hai; leaky bucket traffic ko smoother output mein shape karne ke liye useful concept hai.

10. Algorithm Comparison

Algorithm	Main behavior	Trade-off
Fixed Window	Simple count per fixed interval	Boundary burst
Sliding Window	Rolling interval	More precise, more complexity
Token Bucket	Rate + controlled bursts	State/refill logic
Leaky Bucket	Smooth output rate	Queueing/delay behavior

11. 429 Too Many Requests

Jab client configured rate limit exceed karta hai, HTTP API commonly **429 Too Many Requests** return karti hai.

Request

↓

Rate limiter

↓

limit exceeded

↓

HTTP 429

Optional:

Retry-After: <seconds>

Client ko Retry-After header diya ja sakta hai when appropriate, especially when the server communicates when retrying may be reasonable.

12. Redis Rate Limiting

Distributed Node.js deployment mein Redis useful hai because all API instances can access shared rate-limit state.

Client

LB

■ → Node 1 ■■

```

■→ Node 1 ■■
■→ Node 2 ■■→ Redis

```

 $\rightarrow$ Node 2

 $\rightarrow$ Node 3

Redis:

```
user:123 → rate-limit state
```

13. Local Memory Problem

Node 1:
user:123 = 50

Node 2:
user:123 = 50

Node 3:
user:123 = 50

Global limit becomes difficult
because state is split.

Single Node application mein in-memory limiter possible hai. Multiple instances mein shared state ya centralized/edge rate limiting strategy chahiye.

14. Redis Fixed Window Example

Key:
rate:user:123:minute

Concept:
INCR key
EXPIRE key 60

If count <= 100:
 allow
else:
 reject 429

Important:
increment + expiry handling should be
designed atomically/robustly.

Production implementation mein race conditions, key expiry, retries, clock behavior and Redis availability ko carefully handle karo. Lua scripts or appropriate atomic Redis operations can help combine related logic.

15. Atomicity Problem

Agar rate-limit check aur increment multiple separate commands mein unsafe order se kiye gaye, concurrent requests ke beech race conditions aa sakti hain.

Request A → GET count = 99
Request B → GET count = 99

A → allow
B → allow

Both increment later
→ limit can be exceeded

Atomic Redis operations or server-side scripts can make check/update behavior safer.

16. Rate Limiter Placement

Option 1:
Client → CDN/WAF/API Gateway → Rate Limit → App

Option 2:
Client → LB → Node.js middleware → Redis

Option 3:
Both:
Edge protection + application-specific limits

Coarse global protection edge par, while business/user/endpoint-specific rules application or gateway layer par ho sakte hain.

17. Login Rate Limiting

Authentication endpoints usually stricter limits deserve karte hain because password guessing/credential abuse risk hota hai.

POST /login

Possible dimensions:

IP limit

+

account/user limit

+

endpoint limit

Exceeded:

→ 429

→ monitoring/alerting

Security controls ke liye rate limiting alone sufficient nahi; authentication hardening, MFA, credential protection and abuse detection may also be needed.

18. API-Specific Limits

GET /products

→ 1000 req/min

POST /orders

→ 100 req/min

POST /login

→ 10 req/min

POST /otp

→ stricter policy

Different endpoints ki cost and abuse risk different hoti hai, so one global limit is not always appropriate.

19. Burst Traffic

Burst ka matlab short period mein unusually high requests. Token bucket controlled burst allow kar sakta hai while average refill rate limit rakhta hai.

Limit:

10 req/sec

Bucket:

capacity 50

Short burst:

50 requests → may pass

Then:

tokens refill at 10/sec

20. Rate Limit Headers

APIs rate-limit information communicate karne ke liye headers use kar sakti hain, such as limit, remaining, reset/retry information. Exact header conventions API design par depend karti hain.

Example concept:

Limit: 100

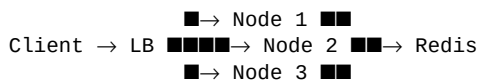
Remaining: 23

Retry-After: 10

Client can adapt request behavior.

21. Distributed Rate Limiting

Multiple application instances ke across one consistent limit enforce karna distributed rate limiting hai.

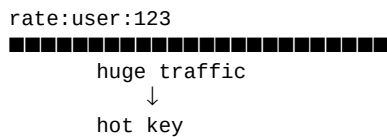


All nodes share limiter state.

Network latency to Redis, Redis availability, hot keys and atomicity become part of the design.

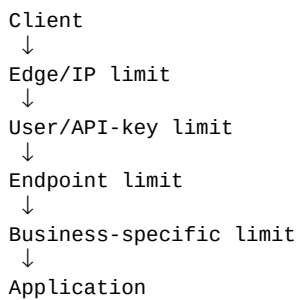
22. Hot Key Problem

Agar millions of requests same Redis key ko hit kar rahe hain, one key can become a hotspot. Distributed cache/cluster architecture mein bhi a single hot key bottleneck ban sakti hai.



Mitigations workload-specific hain: local coarse limits, key distribution, batching, edge limiting, or architecture changes.

23. Multi-Level Rate Limiting



Multiple limits defense-in-depth provide kar sakte hain, but rules too aggressive hue toh legitimate users bhi block ho sakte hain.

24. Rate Limiter Failure — Redis Down

Redis unavailable hone par system ko decide karna hota hai: fail-open (requests allow) ya fail-closed (requests reject). Choice endpoint risk and business requirements par depend karti hai.

Redis DOWN

Option A:
fail-open → API continues
risk → less protection

Option B:
fail-closed → reject
risk → availability impact

Decision = security vs availability trade-off

25. Rate Limiting vs Throttling

Terms sometimes overlap. Rate limiting usually means enforcing a maximum request rate; throttling can also mean slowing/delaying work instead of immediately rejecting it.

Rate limit:
101st request → 429

Throttling:
101st request → queued/delayed
or processed at controlled rate

26. Rate Limiting vs WAF vs DDoS Protection

Layer	Main role
Rate limiter	Control request frequency
WAF	Inspect/filter web traffic based on security rules
DDoS protection	Absorb/mitigate large-scale volumetric/network attacks

27. Production Architecture

Internet
↓
CDN / WAF / API Gateway
↓
Edge/global rate limit
↓
Load Balancer
↓
Node.js × N
↓
Redis-based app-specific rate limit
↓
Business APIs

Monitoring:
429 rate + Redis latency/errors
+ request rate + p95/p99 latency

28. Observability

- Requests/sec
- 429 responses/sec
- Rate-limit rejection percentage
- Redis latency
- Redis errors/timeouts
- Hot keys
- Per-endpoint request rate
- Worker/API CPU and memory
- p50 / p95 / p99 API latency
- Abuse/security alerts

29. Interview Q&A;

Question	Short answer
----------	--------------

Rate limiter kya hai?	Client request frequency ko control karne wala mechanism.
Why use it?	Abuse control, fair usage, overload protection, predictable capacity.
429?	Too Many Requests HTTP response.
Fixed window?	Fixed interval counter.
Sliding window?	Rolling time interval based limit.
Token bucket?	Tokens refill at rate; requests consume tokens; controlled bursts possible.
Leaky bucket?	Work/output ko controlled rate se smooth karta hai.
Why Redis?	Multiple Node instances ke liye shared low-latency state.
Why not local memory?	State instances mein split ho jata hai.
Idempotency needed?	Rate limiting itself is not idempotency, but related distributed request handling may need atomic operations.
Redis down?	Choose fail-open or fail-closed based on security/availability requirements.
Hot key?	One limiter key receives disproportionate traffic.
IP vs user ID?	IP handles unauthenticated clients; user/API-key gives identity-aware limits; often combine.
Rate limiter = DDoS protection?	No. It is one layer; WAF/CDN/DDoS controls may also be required.

30. Practice Scenarios

1. API limit 100 requests/minute hai. Fixed window mein boundary burst kaise possible hai?
2. Design a token bucket for 10 requests/sec with burst capacity 50.
3. Three Node.js instances hain. Global user limit kaise enforce karoge?
4. Redis GET + SET based limiter mein race condition kaise aa sakti hai?
5. Login endpoint ko normal product GET se stricter limit kyun doge?
6. Redis down ho gaya. Fail-open vs fail-closed kab choose karoge?
7. One user key hot ho gaya. What would you investigate?
8. 429 response ke saath client ko kya information useful ho sakti hai?
9. Rate limiter ko LB ke pehle ya Node.js ke andar rakhne ke trade-offs kya hain?
10. Design rate limiting for a public API with per-IP, per-user and per-tenant limits.

31. Final Cheat Sheet

RATE LIMITER CHEAT SHEET

Goal:

- control request rate
- protect capacity
- prevent abuse
- fair usage

Algorithms:

Fixed Window
Sliding Window
Token Bucket
Leaky Bucket

Identity:

IP
User ID
API Key
Tenant
Endpoint

Distributed:

Node × N

↓

Shared Redis / gateway state

Important:

- atomicity
- 429
- Retry-After
- burst handling
- hot keys
- Redis failure strategy
- observability

Golden interview line:

"I'll choose the limiting key and algorithm based on the traffic pattern, then use a shared state store or gateway for distributed enforcement, with 429 handling, atomic updates, monitoring and a defined Redis-failure strategy."

32. Local Practice

Node.js + Express middleware banao. Pehle in-memory fixed-window limiter implement karo. Phir 3 Node processes run karke limitation observe karo. Redis add karke shared limiter banao, atomic update implement karo, 429 +

Retry-After add karo, aur Grafana/Prometheus mein request rate, 429 rate and Redis latency monitor karo.